Chat agent development using Open source LLM

Problem statement: Build a multi-turn chat agent that supports a Remote Patient Monitoring (RPM) workflow for patients.

Constraints

1. Use English as an input language and synthetic data only for training/evaluation.
2. Use only open-source models based on your preference (**≤9B model size**). Mention versions and reason for choosing the model, We can provide a GPU if needed.
3. Timeline: 5 days,

Scenario: Build and evaluate an open-source LLM model that can run a Remote Patient Monitoring (RPM) chat workflow (given below) with safe behavior and tool/function calling. Improve the accuracy of the flow via training the open source LLM model.

The chat workflow must support these steps: (chat examples are attached as test_cases.yaml)

1. **Patient onboarding:** greet patient and verify identity (first, last name, date of birth using tool)
2. **Device setup**: pulse oximeter, BP device, scale, thermometer with tablet to take readings
3. **Troubleshooting** device issues
4. **Simple education**: how to take vitals with all devices
5. **Closing the chat.**

Safety rules: if a user reports red flags (e.g., “chest pain”, “shortness of breath”, “SpO2 88”), do **not** give medical advice, call escalate_to_nurse (reason) tool. **Tool calls** (must be used when required)

1. verify_identity(first_name, last_name, dob)
2. pair_device(device_id): device pairing with tablet
3. start_measurement(device_id, measurement_type)
4. troubleshoot_step(step_id)
5. escalate_to_nurse(reason)

Tasks in the assignment

1. build an open source LLM chat pipeline for multi turn conversations using a hybrid system that combines a local open-source LLM with deterministic control logic with state machine
2. Build a chat interface to have above complete chat flow with agent
3. Create a synthetic dataset to evaluate the above workflow. Report the accuracy, TTFT (time-to-first-token) and total response time for each of the steps mentioned in the workflow. Report latency numbers in GPU/CPU hardware.
4. Create synthetic datasets for SFT and optional preference tuning or any other method to improve the accuracy of workflow.
5. Finetune the LLM with synthetic datasets and report the accuracy. You must evaluate and report results for:
    a. Baseline (no fine-tuning)
    b. SFT or LoRA based training

Submission: Submit one ZIP folder with:

1. code/ (runnable Python code)
2. data/ (used for training, validation, evaluation)
3. outputs/ (accuracy report) and demo video of tasks mentioned above.
4. README.md (how to install/run; single command if possible)

Provide a document which states your pipelines, LLM model selection, accuracy/latency metrics, and any other module for analysis etc with your reasoning.